

SkyTech Code Documentation

Version V1.2 · 2026-10-10 · matches repository release R4.0 · maintained by SkyTech_Manager

Every program file in the SkyTech repository with the comments written in it: what the file is for, what goes in and comes out, how to run it, which SkyTech error codes it can raise, and one line for each function, table, view and procedure. Generated from the code itself, so it stays in step with each release.

1. How the code is commented

Rule	What it means
Version line	First and last line of every file: Version: Vx.y (date) — <path> — Vx.y. Files are edited in place; the CHANGELOG and version register record each change.
File purpose block	Right after the version line: Purpose, Inputs, Outputs, Run, Errors (SkyTech codes) and, where relevant, Security. JavaScript uses <code>/** @file ... */</code> , Python a module docstring, SQL and .bat files comment lines (<code>-- / REM</code>).
Function comments	One line directly above each function, procedure, view or table: name — what it does (and the codes it can raise). JavaScript <code>/** ... */</code> or <code>//</code> , Python docstrings, SQL <code>--</code> lines above CREATE.
Error codes	Code never invents messages: it throws <code>AppError('SKY-...')</code> (Node) or <code>skylog.fail('SKY-...')</code> (Python); SQL THROWS numbers that errors.js maps to codes. All codes live in <code>config/error-codes.json</code> .
Logging	Node: <code>require('./logger')</code> — <code>log.info/warn/error/security(category, message, {code, reqId, ...})</code> . Python: <code>skylog.get(name).info(...)</code> . Never log passwords or tokens (redaction is automatic, but do not rely on it).
Security notes	Any input that reaches SQL, HTML or the file system is validated or escaped at that point, and the comment says how.

2. Files at a glance

Group	Files	Documented functions / objects
Admin site — server	9	107
Admin site — screens	3	23
Admin site — tools	5	11

Group	Files	Documented functions / objects	
Database (SQL Server 2014)	5	29	
Scripts	12	28	
Repository	3	0	
File	Version	Group	
admin-site/server.js	V2.2	Admin site — server	
admin-site/src/security.js	V1.1	Admin site — server	
admin-site/src/roles.js	V1.0	Admin site — server	
admin-site/src/errors.js	V1.1	Admin site — server	
admin-site/src/logger.js	V1.1	Admin site — server	
admin-site/src/schema.js	V1.5	Admin site — server	
admin-site/src/db/mssql.js	V2.2	Admin site — server	
admin-site/src/db/memory.js	V2.2	Admin site — server	
admin-site/src/demo/render.js	V1.2	Admin site — server	
admin-site/public/app.js	V2.3	Admin site — screens	
admin-site/public/style.css	V2.2	Admin site — screens	
admin-site/public/index.html	V2.2	Admin site — screens	
admin-site/scripts/doctor.js	V1.1	Admin site — tools	
admin-site/scripts/create-admin.js	V2.1	Admin site — tools	
admin-site/scripts/build-demos.js	V2.0	Admin site — tools	
admin-site/start-admin.bat	V2.0	Admin site — tools	
admin-site/.env.example	V2.0	Admin site — tools	
sql/01_create_skytechcrm.sql	V2.3	Database (SQL Server 2014)	
sql/02_daily_batch_query.sql	V1.0	Database (SQL Server 2014)	

File	Version	Group
sql/04_admin_site.sql	V1.6	Database (SQL Server 2014)
sql/05_demo_sites_built.sql	V2.2	Database (SQL Server 2014)
sql/06_security_and_logging.sql	V1.1	Database (SQL Server 2014)
scripts/common/skylog.py	V1.0	Scripts
scripts/security/scan-secrets.js	V1.0	Scripts
scripts/leads/build_sample_from_extract.py	V1.3	Scripts
scripts/leads/make_sample.py	V1.3	Scripts
scripts/leads/overture_browser_extract.js	V1.1	Scripts
scripts/docs/build_infrastructure.py	V1.8	Scripts
scripts/docs/build_tech_stack.py	V1.4	Scripts
scripts/docs/build_ops_docs.py	V1.0	Scripts
scripts/prompt-book/build_prompt_book.py	V2.1	Scripts
scripts/prompt-book/content.py	V2.0	Scripts
scripts/website/content.py	V1.0	Scripts
scripts/website/build_demo_sites.py	V2.0	Scripts
push-to-github.bat	V2.0	Repository
.gitignore	V1.1	Repository
config/error-codes.json	V1.3	Repository

3. File by file

Admin site — server

admin-site/server.js V2.2

@file SkyTech Admin site web server.

Purpose: serves the Admin screens (public/) and the JSON API used by them: sign-in, dashboard, drill-down tables, create/edit/delete of SkyTechCRM records, the live duplicate check, demo-site preview/save and the system log. Every change is written to dbo.AuditLog; every request gets a reference number and is logged.

Run: npm start (SQL Server, needs admin-site\.env) | npm run demo (sample data in memory, no database)
 npm run doctor (health check) | npm run create-admin (create or reset a login)

Inputs: admin-site\.env (see .env.example), config/error-codes.json, SkyTechCRM or the demo sample CSV.

Outputs: http://127.0.0.1:3030 (local only by default); logs/runtime/admin-YYYY-MM-DD.log.

Security: see src/security.js (headers, CSRF, rate limits, session timeouts, lockout) and docs/architecture/SkyTech_Security_Architecture. Only whitelisted tables/columns (src/schema.js) reach SQL.

Errors: every failure is answered as { error, code, requestId, hint } using SkyTech codes (src/errors.js); look the code up in docs/architecture/SkyTech_Troubleshooting_Guide.

Function / object	What it does
<code>fatal</code>	prints a start-up error with its code and fix, logs it, and stops the process.
<code>wrap</code>	lets async route handlers pass errors to the central error handler.
<code>auth</code>	the request must come from a signed-in user (SKY-AUTH-004, or SKY-AUTH-003 if the session just expired).
<code>EPOCH</code>	per-login change counter; bumping it ends that login's open sessions (see <code>auth</code>).
<code>refuse</code>	logs and returns SKY-AUTH-005 when the signed-in role lacks a permission (see <code>src/roles.js</code>).
<code>need</code>	the role must have an extra right: users syslog duplicates demoSave remove (<code>src/roles.js</code>).
<code>mayWrite</code>	the role may create/edit records of :entity (runs after <code>entity</code>).
<code>entity</code>	resolves :entity against the whitelist in <code>src/schema.js</code> (SKY-DATA-006 if unknown).
<code>checkDemo</code>	field rules for demo sites before saving: slug format, colours, free-licence photo hosts.
<code>DUMMY_HASH</code>	real bcrypt hash of a random value, compared when the username is unknown (equal timing).

Function / object	What it does
cat	validates the duplicate category (?cat= or body.cat).
dupCompare	loads both sides of a suspected duplicate pair for the side-by-side screen.
PAGE_CSP	demo pages may load only Google Fonts and free-licence photos; no scripts at all.
demoPage	loads a DemoSites record and renders its HTML (SKY-DATA-004 if missing).
confirmMe	re-checks the signed-in admin's own password; wrong attempts count towards the lockout.
checkNewPassword	applies the password policy (SKY-AUTH-006 lists what is missing).
activeAdminsAfter	how many active admins remain if login id gets role/isActive.

```
@file Security controls for the Admin site (one place to review them all).
Purpose: implements the controls described in docs/architecture/SkyTech_Security_Architecture:
1. checkConfig      - refuses to start with unsafe settings (weak SESSION_SECRET, open network without HTTPS).
2. headers          - browser security headers: strict Content-Security-Policy, no framing, no sniffing, HSTS on HTTPS.
3. requestId        - gives every request a reference (X-Request-Id) shown to the user and written to the log.
4. rateLimit        - limits requests per computer (IP) per minute; stricter for writes.
5. originCheck      - blocks state-changing requests sent from other websites.
6. csrf             - per-session anti-forgery token required on every POST/PUT/DELETE.
7. sessionTimeout   - signs users out after SESSION_IDLE_MIN of inactivity and after SESSION_MAX_HOURS.
8. loginGuard       - locks a username after 5 wrong passwords (a computer after 20) for 15 minutes; admins can unlock.
9. passwordPolicy   - rules for new passwords (used by scripts/create-admin.js).
Inputs:  environment settings (see .env.example); logger; AppError.
Errors:  SKY-CFG-002/003 at start; SKY-SEC-001/002/007 and SKY-AUTH-002/003 at run time.
```

Function / object	What it does
checkConfig	validates security-relevant settings before the server starts. Returns { secret, secure } or throws AppError (the server prints it and exits). Demo mode may run without a secret (a random one is generated for that run).
cookieName	"__Host-" prefix on HTTPS (browser-enforced: secure, no domain, path /).
APP_CSP	policy for the Admin screens: only this site's own scripts, no inline script, no framing.
headers	adds security headers to every response; API responses are never cached.
requestId	12-character reference for each request; logged and returned in X-Request-Id and error bodies.
rateLimit	fixed-window limiter in memory, per IP. Defaults: 300 requests/minute overall, 60 writes/minute (RATE_LIMIT_PER_MIN, RATE_LIMIT_WRITES_PER_MIN).
originCheck	rejects writes whose Origin/Referer belongs to another site (defence in depth with SameSite cookies).
issueCsrf	creates (once per session) and returns the anti-forgery token.
csrf	requires header X-CSRF-Token = session token on POST/PUT/DELETE (login is exempt; it has no session yet).
sessionTimeout	idle timeout (SESSION_IDLE_MIN, default 30) and absolute lifetime (SESSION_MAX_HOURS, default 8). Expired sessions are destroyed and the request continues as "not signed in" (SKY-AUTH-003 on protected routes).

Function / object	What it does
loginGuard	counts wrong passwords per username (5) and per computer/IP (20) within 15 minutes; reaching the limit locks that key. check(user, ip) → AppError null; fail(user, ip); ok(user, ip); status(user) → { FailedAttempts, LockedUntil } for the Logins screen; unlock(user) clears a username lock.
passwordPolicy	returns a list of problems (empty = acceptable). Rules: 12+ characters, upper and lower case, a digit, not containing the username, not a common password.

admin-site/src/roles.js V1.0

@file Roles and permissions for the SkyTech Admin site (one place to read and change them).
 Purpose: decides what each role may do. The server checks every request against this table (SKY-AUTH-005 when refused) and sends the signed-in user's permissions to the screens, which hide what they cannot use.
 Roles (match the SkyTech agents' work):
 admin – everything, including Logins and System log (owner).
 manager – all data: create, edit, delete, resolve duplicates, save demo sites; no Logins or System log.
 sales – PhoneMarketing / SocialMediaMarketing work: edit leads, log activities; read everything else.
 webdev – WebsiteDeveloper work: edit demo sites and web presence, save demo pages; read everything else.
 viewer – read only.
 Change a role here, then update docs (Security Architecture "Access control") and the version.

Function / object	What it does
ROLES	key → label, short description, writable screens, and extra rights.
canWrite	may this role create/edit records of this screen (entity key)?
can	may this role use an extra right: remove duplicates demoSave users syslog?
permsFor	the permissions sent to the screens for the signed-in user.
list	role keys, labels and descriptions for the Logins screen.

@file Error codes for the Admin site.

Purpose: one place that turns any failure into a SkyTech error code (SKY-<AREA>-<NNN>) with a plain message, an HTTP status and a fix hint. Codes, titles and fixes come from config/error-codes.json, the same catalog that builds the Troubleshooting Guide, so screen, log and guide always agree.

Inputs: config/error-codes.json (falls back to a tiny built-in list if the file is missing).

Outputs: AppError objects; fromDbError() maps SQL Server error numbers to codes.

Used by: server.js (central error handler), src/db/mssql.js, scripts/doctor.js.

Function / object	What it does
loadCatalog	reads the error catalog once; returns a Map code → entry.
AppError	an error that carries a SkyTech code. code: catalog code; message: text for the user (defaults to the catalog title); details: extra facts for the log only (never shown to the user).
info	returns the catalog entry for a code (or undefined).
fromDbError	converts a SQL Server / driver error into an AppError. Looks at the SQL error number (err.number or err.originalError.info.number) and driver codes.

@file Structured logging for the Admin site and its scripts.

Purpose: writes one JSON line per event to logs/runtime/admin-YYYY-MM-DD.log (one file per day) and a short readable line to the screen. Every line has a time, level, category, SkyTech error code (if any) and the request reference shown to the user, so a problem on screen can be found in the log in seconds.

Security: values of sensitive fields (passwords, hashes, secrets, tokens, cookies) are replaced by "[redacted]" before writing. Logs stay on this computer (folder is git-ignored) and are deleted after LOG_RETENTION_DAYS.

Inputs: env LOG_DIR (default <repo>/logs/runtime), LOG_LEVEL (debug|info|warn|error, default info), LOG_RETENTION_DAYS (default 30).

Outputs: log files; functions debug/info/warn/error/security.

Errors: if the folder cannot be written the logger keeps running and only prints to the screen (SKY-SYS-003).

Function / object	What it does
ensureDir	creates the log folder; on failure switches to screen-only logging (SKY-SYS-003).
redact	deep-copies a value replacing sensitive keys with "[redacted]" (max depth 4 to stay small).
fileFor	today's log file name for a prefix ("admin", "doctor", "scripts").
write	the single place that records an event. level: debug info warn error; category: e.g. http, auth, security, db, demo, startup; msg: short text; fields: extra data (code, reqId, user, ip, status, details...).
prune	deletes log files older than LOG_RETENTION_DAYS (runs at start and once a day).
tail	newest n log entries (admin, doctor and scripts logs of the last 3 days, merged by time) for the System log screen.
security	security events (sign-in, lockout, CSRF, rate limit, permission) always at warn or info.
to	logger that writes to another file prefix (used by doctor and scripts).

@file Whitelist of what the Admin site may read and change (security control: "allow-list"). Only names listed here ever reach SQL, so the site cannot touch other tables or columns, and an unknown table name in a request is refused (SKY-DATA-006). Each entity: table (writes), source view (reads), key, columns with type, label, required/readonly/options, and whether the screen is writable.

Function / object	What it does
STATUSES	lead pipeline steps, in order (also used to keep the "further along" status when merging).
ACTIVITY_TYPES	allowed values for Activities.ActivityType.
AGENTS	SkyTech agent names offered in drop-downs.
editableColumns	the columns a create/update may write (not readonly, not view-only).

@file SQL Server adapter (SkyTechCRM on SQL Server 2014+). Same functions as src/db/memory.js (demo mode).
Security: every value is sent as a typed parameter (no string-built SQL with user input → no SQL injection);
table and column names come only from the whitelist in src/schema.js.
Settings: DB_SERVER, DB_DATABASE, DB_AUTH (windows|sql), DB_DRIVER, DB_USER/DB_PASSWORD, DB_ENCRYPT, DB_TRUST_CERT.
Errors: every driver/SQL error is converted by fromDbError() (src/errors.js) into a SKY-DB-/DUP-/DATA- code.

Function / object	What it does
connect	builds the connection pool: Windows login via msnodesqlv8/ODBC, or SQL login via tedious.
sqlType	maps a schema column type (int, money, date, bit, text...) to the mssql parameter type.
coerce	converts a form value to the column type ("" → NULL, "1"/"true" → bit, numbers parsed).
friendly	converts any SQL Server error into a SkyTech AppError (codes in config/error-codes.json).
q	wraps a whitelisted column/table name in [brackets].
init	opens the connection pool and runs a test query; failures become SKY-DB-00x codes.
_pool	the open connection pool (used by scripts/doctor.js for read-only checks).
ping	health check used by /api/health and doctor.
getUser	active admin user by name (for sign-in); returns hash and role.
touchLogin	records the time of the last successful sign-in.
createUser	adds or resets a login (bcrypt hash only) — used by scripts/create-admin.js.
listUsers	all logins without password hashes, admins first.
_userCols	true once sql/04 V1.6 added the CreatedBy/UpdatedBy columns (checked once, then cached).
getUserById	one login including its hash (used for checks on the server only).
addUser	creates a new login; an existing username → SKY-DUP-001 (unique rule UQ_AdminUsers_Username).
updateUser	changes role and/or active flag of a login.
setPassword	stores a new bcrypt hash for a login.

Function / object	What it does
audit	appends one row to dbo.AuditLog (who, what, which record, JSON details). Append-only (sql/06).
dashboard	KPI numbers and chart data for the Dashboard screen.
list	one page of rows for a drill-down table: search, column filters, sort, paging (all parameterised).
get	one record by its key (from the entity's view or table).
create	inserts a record (only editable whitelisted columns); duplicates → SKY-DUP-001.
update	changes a record's editable columns; returns the updated row.
duplicateSummary	counts from dbo.vw_DuplicateCheck by category and severity (live duplicate badge).
compareCompanies	two companies side by side for the merge screen.
compareLeads	two leads side by side.
webRows	all web-presence rows of one company (duplicate repair).
compareActivities	two activities side by side.
mergeCompanies	EXEC dbo.usp_MergeCompanies (errors 50001/50002 → SKY-DUP-005).
mergeLeads	EXEC dbo.usp_MergeLeads (errors 50011/50012 → SKY-DUP-002).
fixWebPresence	EXEC dbo.usp_FixDuplicateWebPresence (error 50021 → SKY-DUP-003).
removeActivity	deletes one repeated activity.
dismissDuplicate	records "not a duplicate" so the pair is hidden from the check.
remove	deletes a record; linked records block it (SQL 547 → SKY-DATA-003).

@file DEMO adapter: no database needed (npm run demo). Loads the 100-lead sample CSV into memory so the site can be tried and tested. Changes are lost when the server stops. Implements exactly the same functions and the same duplicate rules as src/db/mssql.js, and throws the same SkyTech codes (DUP-001, DATA-002/003/004, DUP-002/003). Inputs: data/samples/SkyTech_Leads_Sample100.csv, demo-sites/designs/*.json.

Function / object	What it does
T	the in-memory tables (same names and columns as SkyTechCRM).
parseCSV	small CSV reader for the sample file (quoted cells, doubled quotes).
normName	same normalisation as dbo.fn_NormName (lower case, "&" → and, no punctuation or Inc/LLC).
digits	phone digits only, without a leading 1 (same as dbo.fn_DigitsOnly).
niche	Real Estate / Utility Trades from the Industry text.
dupCheck	demo version of dbo.vw_DuplicateCheck: Exact / Likely / Possible pairs across all tables.
views	builds the joined "views" (vw_LeadDetail, vw_DemoSiteDetail...) from the tables on each call.
coerce	converts form values to the column type; bad numbers or dates → SKY-DATA-002.
cmp	sort helper that puts empty values first.
init	loads the CSV and demo designs, creates the demo users admin/viewer.
ping	health check; demo data is always available.

@file SkyTech_WebsiteDeveloper page renderer. Turns one DemoSites record (design fields + company facts) into a complete, self-contained one-page website. Used by:

- scripts/build-demos.js (writes demo-sites/<slug>/index.html)
- the Admin site live preview and "Save to folder" (renders straight from the database)

Honesty rules: preview banner + noindex until the business approves; no reviews, prices or licence claims; concept logo and original illustrations; photos only from free-licence hosts with a credit line.

Security: every text value is HTML-escaped (e); colours must be #RRGGBB (hex); fonts from a fixed list (font); photo links only <https://images.pexels.com> or images.unsplash.com, or a local img/ file (img) – anything else is dropped, which prevents script or tracking injection through the design fields (see SKY-SEC-005).

Function / object	What it does
TEMPLATE_VERSION	page template version written into each demo and dbo.DemoSites.TemplateVersion.
e	HTML-escape any text placed in the page.
hex	accepts only #RRGGBB colours, else the default d.
font	accepts only fonts from FONTS (Google Fonts with system fallbacks).
FALLBACK	system fonts used when Google Fonts cannot load (offline: SKY-DEMO-005).
stack	CSS font-family list for a font name.
pick	value if it is in the list, else default.
lines	multi-line field → trimmed non-empty lines (Services, Highlights, Steps).
tel	phone number → +1XXXXXXXXXX for tel: links.
sized	asks Pexels for a compressed copy at width w (faster pages).
mix	lighter tint of a colour (used for soft backgrounds).
KEYWORDS	picks an icon for a service from words in its title.

Admin site — screens

admin-site/public/app.js V2.3

@file SkyTech Admin site front end (plain JavaScript, no framework). Routes: #dashboard, #syslog, #users, #<entity>?search=&f_Col=val

Security: no inline scripts (strict CSP); every value shown is escaped (esc); every change sends the CSRF token (X-CSRF-Token) received at sign-in; buttons appear only for what the role may do (PERMS from /api/me; the server checks again).

Errors: server errors arrive as { error, code, requestId, hint } and are shown as "message (CODE, ref ID)"; the ref ID finds the matching line in logs/runtime/admin-<date>.log and the System log screen.

Function / object	What it does
esc	HTML-escape text before inserting it into the page (prevents script injection).
api	calls the Admin site API. Sends the anti-forgery token (X-CSRF-Token) on every change. On failure throws an Error whose message ends with the SkyTech code and reference, e.g. "Wrong username or password. (SKY-AUTH-001, ref 3fa2...)" — look the code up in the Troubleshooting Guide. If the token is stale (SKY-SEC-001) it refreshes it once and retries automatically.
toast	short message at the bottom of the screen.
cell	formats one table cell by column type (status badge, money, date, link, yes/no).
showLogin	shows the sign-in form (optionally with a message, e.g. session expired).
start	after sign-in: loads /api/me (user, role, CSRF token, version) and the screen metadata, then routes.
checkDuplicates	refreshes the live duplicate badge from /api/duplicates/summary.
watchDuplicates	re-checks duplicates every minute and after each change.
canWrite	true when this role may change records of the screen (the server enforces the same rule: SKY-AUTH-005).
go	navigates to a screen with filters.
route	draws the screen for the current #route.
miniTable	small clickable table used on the dashboard.
openRecord	opens the record drawer: view/edit/create/delete, plus related records.

Function / object	What it does
renderRelated	lists linked records (company → lead, web presence, activities, demo site).
dupPanel	the Possible duplicates panel: counts by category and severity.
wireDupPanel	click handlers for the duplicate panel buttons.
openCompare	side-by-side compare with Merge / Fix / Not a duplicate actions.
POLICY	shown under every new-password box (the server enforces the same rules, SKY-AUTH-006).
pwRow	a password input with a show/hide toggle.
openForm	shows a small form in the drawer; onSave(values) runs on submit, errors stay in the form.
renderUsers	the Logins screen: every login with role, status, lock state, created/changed by and last sign-in.
renderSyslog	shows /api/health and /api/logs; level filter; each code opens its guide entry.
closeDrawer	hides the side panel (record editor / compare view).

admin-site/public/style.css V2.2

Styles for the SkyTech Admin site. Colours are variables in :root (change them once here).
Sections: layout (sidebar, top bar), tables and badges, record drawer and forms, duplicate panel and compare, dashboard tiles/charts, System log (tr.lg-warn / tr.lg-error rows), sign-in screen, toast messages.
Loaded from the site itself only (Content-Security-Policy style-src 'self').

admin-site/public/index.html V2.2

Admin site page shell: sign-in form, sidebar menu (Dashboard, tables, Possible duplicates, Demo sites, System log for admins), record drawer and toast. Loads style.css, vendor Chart.js and app.js only (no inline script, CSP).

admin-site/scripts/doctor.js

V1.1

@file Health check ("doctor") for the SkyTech Admin site, database and repository.

Purpose: finds the problems listed in the Troubleshooting Guide before they bite, and prints each one with its SkyTech code and the manual fix. Safe to run any time; it only reads (it never changes data or files, apart from writing its own log).

Run: `npm run doctor` full check (needs admin-site\.env and SQL Server)
 `npm run doctor -- --demo` skip the database checks
 `npm run doctor -- --new-secret` print a strong SESSION_SECRET to paste into .env

Outputs: a ✓ / △ / ✖ list on screen and logs/runtime/doctor-YYYY-MM-DD.log. Exit code 1 if any ✖.

Checks: Node version, packages, .env and secrets, network exposure, git hygiene (locks, remote, .env not tracked), log folder, demo-sites folder, error catalog, disk space, port, and in SkyTechCRM: connection, required objects, unique rules, admin logins, recent database errors, app role, last backup.

Function / object	What it does
add	records one check result: level ok warn fail, the SkyTech code (if any), message and fix.
git	runs a git command in the repository; returns trimmed output or null if git is unavailable.
checkLocal	everything that does not need the database.
REQUIRED	database objects the Admin site and scripts rely on (type U=table, V=view, P=procedure, FN=function).
checkDatabase	connects to SkyTechCRM and checks objects, rules, logins, errors, role and backups.

admin-site/scripts/create-admin.js V2.1

@file Create or reset an Admin site login.

Purpose: asks for a username, password (typed hidden) and role, checks the password policy (src/security.js passwordPolicy: 12+ chars, upper/lower case, a digit, not the username, not common), stores a bcrypt hash (cost 12) in SkyTechCRM.dbo.AdminUsers and writes an audit entry.

Run: npm run create-admin (re-run with the same username to reset a password)

Inputs: admin-site\.env database settings.

Errors: SKY-AUTH-006 password policy; SKY-DB-00x database problems; SKY-DATA-002 bad username/role.

Function / object	What it does
ask	prompts on the console; hidden=true shows * instead of the typed characters.

admin-site/scripts/build-demos.js V2.0

```
@file SkyTech_WebsiteDeveloper: builds demo websites from design files and writes the database update.
Inputs : demo-sites/designs/<slug>.json          (unique design + wording for one business)
         data/samples/SkyTech_Leads_Sample100.csv (public facts: phone, email, address)
Outputs: demo-sites/<slug>/index.html             (self-contained page)
         demo-sites/manifest.csv                 (one row per demo, never duplicated)
         sql/05_demo_sites_built.sql             (adds each demo to dbo.DemoSites and marks the lead DemoBuilt; safe
to re-run)
         logs/runtime/scripts-YYYY-MM-DD.log      (what was built, and any error code)
Run      : (from the admin-site folder)  npm run build-demos          (all designs)
                                                npm run build-demos -- c-and-a-plumbing-20723  (one or more slugs)
Errors   : SKY-DEMO-001 design file not valid JSON / missing fields; SKY-DEMO-002 design has no matching lead;
          SKY-DEMO-003 unknown slug; SKY-LEAD-001 sample CSV missing; SKY-SYS-002 disk full.
After the demos are in SkyTechCRM, edit them in the Admin site (Demo sites tab); edits there win.
```

Function / object	What it does
fail	prints the SkyTech code, message and first fix step, logs it, and stops with exit code 1.
REQUIRED	fields every design file must have (SKY-DEMO-001 if one is missing).
parseCSV	small RFC-4180 CSV reader (quoted cells, doubled quotes, CRLF); returns an array of row objects.
sq	SQL literal: NULL for empty, otherwise N'...' with quotes doubled (prevents SQL injection in the generated file).
csvCell	quotes a manifest cell when it contains a comma, quote or line break.

admin-site/start-admin.bat V2.0

```
Purpose : starts the SkyTech Admin site on http://127.0.0.1:3030 (this computer only).
Steps  : 1) installs packages on first use 2) stops if .env is missing (SKY-CFG-001)
        3) runs "npm run doctor" - shows problems with SkyTech codes and fixes; stops on a red X
        4) opens the browser and starts the server (log: ..\logs\runtime\admin-YYYY-MM-DD.log)
Errors  : see docs\architecture\SkyTech_Troubleshooting_Guide.html
```

admin-site/.env.example V2.0

Copy this file to ".env" and fill it in. Never commit .env to GitHub (it is git-ignored and the push scan blocks it). After any change run: `npm run doctor` (checks every setting and prints SkyTech codes with fixes)

---- Web server ----

32+ random characters. Create one with: `npm run doctor -- --new-secret` (SKY-CFG-002 if weak)

---- Sessions and sign-in protection ----

---- Logging (logs/runtime/admin-YYYY-MM-DD.log, JSON lines, passwords redacted) ----

---- Database: SkyTechCRM on SQL Server 2014 Developer ----

---- Demo mode (no database): run "`npm run demo`"; sign in as admin / the password below ----

sql/01_create_skytechcrm.sql V2.3

Purpose : creates the SkyTechCRM database: lead tables, duplicate protection, the lead import procedure, the DemoSites table and the database error log.

Run : SSMS → open this file → Execute (F5). Order: 01 → lead import → 04 → 05 → 06.

Errors : PRINT 'WARNING: duplicate...' = SKY-DUP-004 (fix duplicates in the Admin site, run again); failures inside procedures are written to dbo.ErrorLog (SKY-DB-010) and re-raised.

SkyTechCRM setup + duplicate protection. SQL Server 2014 Developer (also newer versions).

SAFE TO RE-RUN: creates what is missing, upgrades a V1 database, never deletes data.

How duplicates are prevented

1. Every company is matched on its source ID (e.g. Overture ID), else on normalized name + ZIP. Unique indexes enforce both rules in the database.
2. One WebPresence row and one Lead per company (unique on CompanyID).
3. Imports go through staging (dbo.StgLeads) + dbo.usp_ImportStagedLeads, which UPDATES existing companies and INSERTS only new ones. Re-running an import changes nothing new.
4. Every import run is logged in dbo.ImportBatches (rows in / inserted / updated / skipped).
5. dbo.vw_PossibleDuplicates lists near-duplicates (same phone, or same name in another ZIP) for a person to review.

How errors are recorded (V2.3)

Every procedure has TRY/CATCH: on failure it rolls back, calls dbo.usp_LogError (one row in dbo.ErrorLog with procedure, error number, line, message, user and time) and re-raises the error to the caller.

Read it with: SELECT TOP 20 * FROM dbo.ErrorLog ORDER BY ErrorLogID DESC;

Function / object	What it does
TABLE dbo.ImportBatches	—
TABLE dbo.Companies	—
TABLE dbo.WebPresence	—
TABLE dbo.Leads	—
TABLE dbo.Activities	—

Function / object	What it does
TABLE dbo.StgLeads	—
FUNCTION dbo.fn_DigitsOnly	—
FUNCTION dbo.fn_NormName	—
VIEW dbo.vw_PossibleDuplicates	—
TABLE dbo.ErrorLog	dbo.ErrorLog: one row per failure caught inside a SkyTech procedure (never holds passwords or lead data).
PROCEDURE dbo.usp_LogError	dbo.usp_LogError: call ONLY inside a CATCH block (after ROLLBACK). Records ERROR_*() details; never fails itself.
PROCEDURE dbo.usp_ImportStagedLeads	dbo.usp_ImportStagedLeads: moves rows from dbo.StgLeads into Companies/WebPresence/Leads. Updates companies it already has (same source ID, else same name + ZIP), inserts only new ones, records the run in dbo.ImportBatches, empties staging. All-or-nothing: any failure rolls back, is logged (SKY-IMP-002) and re-raised.
TABLE dbo.DemoSites	One row per company (unique), edited in the Admin site "Demo sites" tab.

sql/02_daily_batch_query.sql V1.0

BackOffice agent: 20 unchecked companies in one industry and city

sql/04_admin_site.sql V1.6

Purpose : objects used by the SkyTech Admin site: sign-in users, audit log, detail views for the screens, the live duplicate check and the three duplicate-repair procedures. SQL Server 2014 Developer (SSMS).

Run : AFTER sql/01_create_skytechcrm.sql (V2.3+, which creates dbo.usp_LogError). SAFE TO RE-RUN; never deletes data.

Errors : THROW 50001/50002 → SKY-DUP-005, 50011/50012 → SKY-DUP-002, 50021 → SKY-DUP-003 (shown in the Admin site); unexpected failures are rolled back, written to dbo.ErrorLog (SKY-DB-010) and re-raised.

Function / object	What it does
TABLE dbo.AdminUsers	dbo.AdminUsers: Admin site sign-ins. Passwords are stored only as bcrypt hashes (cost 12), never in plain text. Role: admin, manager, sales, webdev or viewer (see V1.6 below). Manage logins on the Admin site Logins screen (or with npm run create-admin when no admin can sign in).
TABLE dbo.AuditLog	dbo.AuditLog: who changed what and when (sign-ins, creates, edits, deletes, merges). Made read-only by sql/06 (SKY-SEC-006).
VIEW dbo.vw_LeadDetail	—
VIEW dbo.vw_WebPresenceDetail	—
VIEW dbo.vw_ActivityDetail	—
TABLE dbo.DuplicateDismissals	—
VIEW dbo.vw_DuplicateReview	—
PROCEDURE dbo.usp_MergeCompanies	—
VIEW dbo.vw_DuplicateCheck	—
PROCEDURE dbo.usp_MergeLeads	—
PROCEDURE dbo.usp_FixDuplicateWebPresence	—
VIEW dbo.vw_DemoSiteDetail	—

sql/05_demo_sites_built.sql V2.2

Generated by admin-site/scripts/build-demos.js (SkyTech_WebsiteDeveloper). Safe to re-run; never creates duplicates.

1) Adds each demo to dbo.DemoSites (one row per company). Rows that already exist are left alone, so edits made in the Admin site win.

2) Marks the lead DemoBuilt (only if it is still New / Checked / NoSite) and logs one Note activity per demo.

Run in SSMS after sql/01 (V2.3) and sql/04 (V1.5); then run sql/06. All-or-nothing: a failure rolls back, is written to dbo.ErrorLog (SKY-IMP-001) and shown in the Messages tab. Demos listed under NotFoundInDatabase need their lead imported first (SKY-IMP-001).

Change : V1.1 – DENY on usp_BackupSkyTechCRM moved below its CREATE (V1.0 gave Msg 15151 on first run).

Purpose : database security and resilience for SkyTechCRM (SQL Server 2014 Developer, SSMS).

1. SkyTechApp role – least privilege: read/write data and run procedures; no table changes, no audit edits.
2. Optional SQL login for the Admin site (commented out; Windows login stays the default and recommended).
3. Audit log protection – dbo.AuditLog can be added to but never edited or deleted (SKY-SEC-006, error 50100).
4. dbo.usp_BackupSkyTechCRM – full, compressed, checksummed backup + verification (SKY-BAK-001 / SKY-BAK-003).
5. dbo.usp_PurgeErrorLog – removes old rows from dbo.ErrorLog (keeps 90 days by default).
6. Optional SQL Server Audit of failed logins and permission changes (commented out).

Run : AFTER 01 → lead import → 04 → 05. Execute as the owner (your Windows login). SAFE TO RE-RUN; never deletes data.

Errors : every failure is written to dbo.ErrorLog and explained in docs/architecture/SkyTech_Troubleshooting_Guide.

Function / object	What it does
ROLE SkyTechApp	SkyTechApp: what the Admin site needs and nothing more. Schema-level grants cover new tables automatically. No ALTER/CREATE/DROP rights, so a bug or an attacker using the app cannot change the database design.
TRIGGER dbo.trg_AuditLog_Protection	CREATE USER SkyTechAdminSite FOR LOGIN SkyTechAdminSite; ALTER ROLE SkyTechApp ADD MEMBER SkyTechAdminSite; trg_AuditLog_Protect: blocks UPDATE and DELETE on dbo.AuditLog for everyone (even the owner) with error 50100. To archive very old audit rows, an owner must first run: DISABLE TRIGGER dbo.trg_AuditLog_Protect ON dbo.AuditLog; (and ENABLE it again straight after). That action itself shows in the SQL Server log.
PROCEDURE dbo.usp_BackupSkyTechCRM	COMPRESSION, then RESTORE VERIFYONLY (proves the file can be restored). Creates the folder if missing. Run weekly (doctor warns after 7 days, SKY-BAK-003): EXEC dbo.usp_BackupSkyTechCRM; Restore (SKY-BAK-002), in SSMS as owner: RESTORE DATABASE SkyTechCRM FROM DISK = N'C:\SkyTechBackups\<file>.bak' WITH REPLACE, CHECKSUM; Copy the .bak files off this computer (USB drive or cloud folder); *.bak is git-ignored and never goes to GitHub.
PROCEDURE dbo.usp_PurgeErrorLog	dbo.usp_PurgeErrorLog: deletes ErrorLog rows older than @KeepDays (default 90). Owner use; returns rows removed.

Scripts

scripts/common/skylog.py V1.0

Version: V1.0 (2026-10-10) – scripts/common/skylog.py – V1.0

Shared error codes and logging for the SkyTech Python scripts (lead builders, document builders, prompt book).

Purpose : the same SKY-<AREA>-<NNN> codes and log format as the Admin site, so a failure in any script can be looked up in docs/architecture/SkyTech_Troubleshooting_Guide.html.

Use : import skylog (scripts add the scripts/common folder to sys.path first)
log = skylog.get("build_tech_stack")
log.info("wrote file", path=out)
skylog.fail("SKY-DOC-002", "Cannot read the diagram source", path=p) # prints code + fix, exits 1
with skylog.guard("build_tech_stack"): main() # turns any crash into a coded, logged failure
skylog.install("build_tech_stack", "SKY-DOC-002") # same, for scripts written as plain top-level code

Outputs : logs/runtime/scripts-YYYY-MM-DD.log (one JSON object per line; secrets redacted).

Errors : if the log folder cannot be written, logging continues on screen only (SKY-SYS-003).

Function / object	What it does
<code>_catalog</code>	Loads config/error-codes.json once; returns {code: entry} (empty if the file is missing).
<code>_redact</code>	Replaces values of sensitive keys (passwords, tokens...) with [redacted].
<code>Logger</code>	Writes JSON lines to logs/runtime/scripts-<date>.log and short lines to the screen.
<code>write</code>	Records one event. level: info warn error.
<code>get</code>	Returns the logger for a script name (one per name).
<code>fail</code>	Logs the failure with its SkyTech code, prints the first fix step and the guide link, and exits with code 1.
<code>guard</code>	Wraps a script's main work: missing modules → SKY-DOC-001, missing files → SKY-DOC-002 (or the given code), disk full → SKY-SYS-002, anything else → the given code. The full traceback goes to the log only.
<code>install</code>	For scripts written as top-level code: any uncaught error becomes a coded, logged failure (see guard) and a start line is logged. Call once, right after the imports.

scripts/security/scan-secrets.js V1.0

@file Secret scanner run before every push (push-to-github.bat calls it).
Purpose: stops passwords, API keys, private keys or a committed .env from reaching GitHub.
Inputs: the files git tracks plus files staged for the next commit (git ls-files).
Outputs: a list of suspicious lines (value masked) and exit code 1 (SKY-SEC-004) if anything is found; 0 if clean.
Run: node scripts/security/scan-secrets.js (from the repository folder)
Allow: a line containing "scan-secrets: allow" is skipped (use only for documented example values).

Function / object	What it does
RULES	what counts as a secret. Each has a name and a regular expression.
listFiles	files git tracks or has staged; exits with SKY-GIT-001 if git is not available.

scripts/leads/build_sample_from_extract.py V1.3

Version: V1.3 (2026-10-10) – scripts/leads/build_sample_from_extract.py – V1.3 (error codes + logging)
SkyTech_BackOffice lead builder.
Purpose : turns data/raw/overture/<date>/sample100_extract.json (collected in Overture Explorer) into
data/samples/SkyTech_Leads_Sample100.csv (for review) and _import.sql (for SkyTechCRM). Adds review flags and
an A/B/C priority so the team calls the best leads first, and merges the BackOffice "no website"
verification.
Inputs : extract JSON (argument 1) – rows of [name, niche, category, address, locality, county, zip, phone, email,
website, hasFacebook, confidence, overtureId]; optional data/verification/2026-10-02_backoffice_results.txt.
Outputs : CSV + import SQL (loads through dbo.StgLeads → dbo.usp_ImportStagedLeads, so re-running never duplicates).
Run : python scripts/leads/build_sample_from_extract.py [extract.json] (from the repository folder)
Errors : SKY-LEAD-001 input missing/unreadable, SKY-LEAD-002 extract empty; log: logs/runtime/scripts-<date>.log.

scripts/leads/make_sample.py V1.3

Turn Overture Maps Explorer downloads (GeoJSON/CSV) into a 100-lead sample for Baltimore City + Baltimore County + Howard County: real estate and utility trades. Version: V1.3 (2026-10-10) - error codes + logging (V1.2: Baltimore City added). NOTE: its SQL output is superseded; for database imports use scripts/leads/build_sample_from_extract.py, which loads through the duplicate-safe dbo.usp_ImportStagedLeads.

Function / object	What it does
g	Safe nested lookup in Overture properties: g(p, "names", "primary").
load	Yields place properties from a GeoJSON/JSON or CSV download (JSON-in-text fields decoded).

scripts/leads/overture_browser_extract.js V1.1

Purpose: SkyTech_BackOffice helper – reads business places (Overture Maps, free licence) shown on the public explorer map.
Inputs : map tiles already loaded in your browser; Outputs: window.__sky rows → save as sample100_extract.json.
Errors : empty result = SKY-LEAD-002 (zoom in until the places layer shows, keep the map visible, run grab again).
Rules : manual, owner-run, read-only; no automated scraping of Google Maps, Yelp or Facebook.
Run in the browser console on <https://explore.overturemaps.org> (the page exposes `map`).
For each area: grab(name, lat, lon). Results collect in window.__sky; then build the sample.
Note: tiles load only while the map is visible on screen.

scripts/docs/build_infrastructure.py V1.8

Version: V1.8 (2026-10-10) – scripts/docs/build_infrastructure.py – V1.8
Builds docs/architecture/SkyTech_Infrastructure.html (diagram + roles + versions).
Edit the DATA section and re-run to publish a new version.
Also imported by build_tech_stack.py, which reuses SVG and CSS so both documents always show the same diagram.
Run : python scripts/docs/build_infrastructure.py (from the repository folder)
Errors : SKY-D0C-001 missing Python package, SKY-D0C-002 file problem; log: logs/runtime/scripts-<date>.log.

Function / object	What it does
box	Draws one component box (title + up to 3 lines); kind: normal acc (SkyTech part) plan (dashed, planned) store person.
zone	Draws a labelled background zone (Owner PC, Cloud, Internet...).
arrow	Draws an arrow along the points, with optional label, step number and dashed style for planned flows.
table	HTML table helper for the roles/components/versions sections.

scripts/docs/build_tech_stack.py V1.4

Version: V1.4 (2026-10-10) – scripts/docs/build_tech_stack.py – V1.4
Builds docs/architecture/SkyTech_Technology_Stack_Use.html (and a .md copy for the Claude project and GitHub): the "Technology Stack Use" document.
It explains, for a non-technical reader, every tool and technology the SkyTech program uses, and merges them with the architecture diagram (taken from build_infrastructure.py, so both documents always match).
Edit the DATA section, bump DOC_VERSION and add a CHANGELOG line, then run from the repository folder:
python scripts/docs/build_tech_stack.py
Errors: SKY-D0C-001 missing Python package, SKY-D0C-002 file problem; log: logs/runtime/scripts-<date>.log.

Function / object	What it does
journey_svg	Six-step "how SkyTech works" picture (find → check → build → contact → win → run), with the tools used at each step.
stack_tables	One table per tool group (what it is, what we use it for, cost, where it runs, status).

Version: V1.0 (2026-10-10) – scripts/docs/build_ops_docs.py – V1.0

Builds the three R4.0 operations documents in docs/architecture (each as .html and .md):

1. SkyTech_Troubleshooting_Guide – every SkyTech error code (from config/error-codes.json), predicted errors, how logging works, first aid, database error queries. One anchor per code (#SKY-DB-001 ...) so the Admin site System log and every error message can link to it.
2. SkyTech_Security_Architecture – threat model, data classification, trust boundaries, controls mapped to OWASP ASVS / NIST CSF 2.0 / CIS Controls v8, secrets, backup, incident response.
3. SkyTech_Code_Documentation – the comments written in every program file, collected into one reference: file purpose blocks plus one line per function / procedure.

Run : python scripts/docs/build_ops_docs.py (from the repository folder; no extra packages needed)

Inputs : config/error-codes.json, the source files listed in CODE_FILES, the DATA sections below.

Outputs : docs/architecture/<name>.html and .md. PDFs are printed from the HTML (SkyTech_Manager does this with a headless browser; any browser's Print → Save as PDF gives the same layout).

Errors : SKY-DOC-002 when a source file cannot be read; log: logs/runtime/scripts-<date>.log.

Function / object	What it does
page	Wraps body HTML in the standard SkyTech document page (title, version line, footer).
table	HTML table; cells are escaped unless they start with '<' (already HTML).
md_table	Markdown table (pipes in cells replaced so the table never breaks).
write	Writes the .html and .md versions of one document.
code_card	One error-code card with an anchor equal to the code.
boundary_svg	Trust-boundary picture: three zones and the five boundaries B1–B5 (see the table under it).
extract	Returns (header_text, [(name, comment)]) for one source file, following the conventions above.

scripts/prompt-book/build_prompt_book.py V2.1

Version: V2.1 (2026-10-10) – scripts/prompt-book/build_prompt_book.py – V2.1
Builds the SkyTech Prompt Book PDF from content.py.
Run from the repository folder: `python scripts/prompt-book/build_prompt_book.py`
Needs : `pip install reportlab` (free). Errors: SKY-DOC-001 package missing, SKY-DOC-002 file problem.

Function / object	What it does
<code>fmt</code>	Escapes a prompt and puts the ROLE:/TASK:/... keywords in bold on their own lines.
<code>tbl</code>	Styled ReportLab table (header row, grid, wrapped cells).
<code>footer</code>	Page footer: version line and page number on every page.

scripts/prompt-book/content.py V2.0

Version: V2.0 (2026-10-02) – scripts/prompt-book/content.py – V2.0
Text of the SkyTech Prompt Book. Each entry: (title, consolidates, prompt).
"consolidates" says which original messages were merged into the one professional prompt.

scripts/website/content.py V1.0

Version: V1.0 (2026-10-02) – scripts/website/content.py – V1.0
SkyTech_WebsiteDeveloper: wording and colours for each kind of business.

Rules for this file (owner-approved honesty rules, see agents/SkyTech_WebsiteDeveloper.md):

- No claims we cannot prove: no "licensed", "insured", "24/7", "years of experience", "free estimates", awards, prices or reviews. The owner of the business adds those.
- Service lists are typical for the trade and are shown as sample text until the owner confirms.

scripts/website/build_demo_sites.py V2.0

Version: V2.0 (2026-10-05) – scripts/website/build_demo_sites.py – V2.0
Retired in R3.2. Demo websites are now built from demo-sites/designs/*.json by the Admin site tooling,
so every page matches the DemoSites table and the Admin live preview:

```
cd admin-site
npm run build-demos
```

This file stays only for version history; running it changes nothing.

Repository

push-to-github.bat V2.0

Purpose : uploads all saved releases (commits + tags) to the private GitHub repository.
Steps : 1) checks for git lock files left by a crashed git (SKY-GIT-002)
2) runs the secret scan - stops if a password, key or .env would be uploaded (SKY-SEC-004)
3) git push of branch main, then of all release tags (R1.0, R2.0 ...)
Errors : SKY-GIT-005 git missing, SKY-GIT-002 lock file, SKY-GIT-001 sign-in, SKY-GIT-004 GitHub newer, SKY-SEC-004
secret found.
Every code is explained in docs\architecture\SkyTech_Troubleshooting_Guide.html
Run : double-click after SkyTech_Manager reports a new release.
--- 1. lock files -----
--- 2. secret scan (needs Node.js) -----
--- 3. upload -----

.gitignore V1.1

```
Files git must never upload to GitHub.  
Python caches and system files  
Node packages (re-created by npm install)  
SQL Server backups and local database files stay off GitHub  
Secrets: real settings live only in admin-site/.env on this computer  
Runtime logs (may contain user names and IP addresses; kept 30 days on this computer only)  
Temporary transfer files
```

config/error-codes.json V1.3

```
Error catalog V1.3: 67 codes, format SKY-<AREA>-<NNN>. Fields per code: code, area, severity, title, where, causes,  
fix, log, predicted, http, user.
```

Version: V1.2 (2026-10-10) — docs/architecture/SkyTech_Code_Documentation.html — V1.2. Rebuild with: python scripts/docs/build_ops_docs.py